

СЕМИНАРСКА РАБОТА ПО ПРЕДМЕТОТ КОНТИНУИРАНА ИНТЕГРАЦИЈА И ИСПОРАКА

Тема: Image compressor app



Факултет за информатички науки и компјутерско инженерство

Ментор:

проф. д-р Панче Рибарски

Изработил:

Никола Јорданоски 223157

Скопје, Јуни 2025

Содержина

1. Вовед.....	3
2. Dockerfiles и Docker compose	3
3. CI/CD (Github actions)	5
4. Kubernetes	7
5. Корисни линкови	9

Вовед

Апликацијата **Image Compressor** е развиена во рамки на предметот **Континуирана интеграција и испорака**, со цел практично да се применат современи DevOps практики. Проектот опфаќа целосна имплементација на следниве аспекти: поставување на изворниот код на јавен Git репозиториум, докеризација на апликацијата и базата, оркестрација со Docker Compose, конфигурација на CI/CD pipeline со GitHub Actions, и креирање на соодветни Kubernetes манифести за оркестрација на истата.

Апликацијата е составена од три главни сервиси:

- **Backend**, изработен со Flask – кој обезбедува REST API за компресија на слики со соодветните алгоритми.
- **Frontend**, изработен со Vue.js (+ Vite) – кој овозможува корисничка интеракција за прикачување и спуштање на сликите.
- **PostgreSQL** база на податоци – која чува логови за компресираните слики.

Dockerfiles и Docker compose

Backend Dockerfile

Backend-от е развиен во **Python (Flask)** и е пакуван во **python:3.9-slim image**.

Се поставува работна папка `/app`. Се копира `requirements.txt` и се инсталираат зависностите со `pip`. Потоа се копира остатокот од кодот и се експонира портата `5000`. Апликацијата се стартува со `python app.py`.

Frontend Dockerfile

Frontend-от е изграден со **Vue.js** и е пакуван во два чекори:

1. Build Stage: се користи **node:22-slim** за инсталација на зависности со `npm install` и градење на продукцискиот build (`npm run build`).
2. Production Stage: готовата содржина се сервира преку **nginx:alpine**, со сопствена `nginx.conf` конфигурација. Се експонира порта `80` и nginx се стартува како веб сервер.

Овој пристап обезбедува оптимизирана слика и одвоена фаза на build и продукција.

```
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file first for better caching
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```

Слика 1. Dockerfile (backend)

```
FROM node:22-slim as build-stage

# Set working directory
WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy source code
COPY . .

# Build the application
RUN npm run build

# Production stage
FROM nginx:alpine as production-stage

# Copy built assets from build stage
COPY --from=build-stage /app/dist /usr/share/nginx/html

# Copy custom nginx config
COPY nginx.conf /etc/nginx/nginx.conf

# Expose port 80
EXPOSE 80

# Start nginx
CMD ["nginx", "-g", "daemon off;"]
```

Слика 2. Dockerfile (frontend)

Docke Compose

Со **docker-compose.yml** се дефинираат и поврзуваат сите сервиси во заедничка мрежа app-network. Сервиси:

- **database:** PostgreSQL (официјален image) база со иницијализација од *init-db.sql*. Податоците се зачувуваат во именуван волумен *postgres_data*. Контејнерот има healthcheck за проверка на спремност.
- **backend:** Flask backend-от се гради од сопствениот Dockerfile. Преку *environment* променливи се конфигурира врската со базата и ја експонира апликацијата на порта 5000. Сервисот зависи од database, односно се стартува дури откако базата ќе биде спремна.
- **frontend:** Vue frontend-от се гради од сопствениот Dockerfile. Сервисот зависи од backend и ја експонира апликацијата на порта 3000, која се мапира на порта 80 во контејнерот.

Сите сервиси се конфигурирани да се рестартираат при пад (restart: unless-stopped), и се поврзани во заедничка bridge мрежа, што овозможува лесна комуникација помеѓу нив.

```

services:
  # PostgreSQL Database
  database:
    image: postgres:15-alpine
    container_name: image-compressor-db
    environment:
      POSTGRES_DB: image_compressor
      POSTGRES_USER: admin_user
      POSTGRES_PASSWORD: admin
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./backend/init-db.sql:/docker-entrypoint-initdb.d/init-db.sql
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin_user -d image_compressor"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app-network

# Flask Backend
backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: image-compressor-backend
  environment:
    DATABASE_URL: postgresql://admin_user:admin@database:5432/image_compressor
    PORT: 5000
  ports:
    - "5000:5000"
  depends_on:
    database:
      condition: service_healthy
  networks:
    - app-network
  restart: unless-stopped

# Vue.js Frontend
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: image-compressor-frontend
  ports:
    - "3000:80"
  depends_on:
    - backend
  networks:
    - app-network
  restart: unless-stopped

volumes:
  postgres_data:

networks:
  app-network:
    driver: bridge

```

Слика 3. Docker-compose.yaml

CI/CD (Github actions)

За автоматизирана изградба и објава на Docker сликите, во проектот е конфигуриран CI процес со помош на **GitHub Actions**. Целта е при секој push или pull request кон главната гранка (master), автоматски да се изградат и испратат нови Docker слики на **Docker Hub** за backend и frontend.

Pipeline

CI процесот е дефиниран во фајлот main.yaml, и се активира на следниве настани:

- push кон master гранка
- pull_request кон master гранка

Pipeline-от содржи неколку последователни чекори:

1. **Checkout**: Се превзема кодот од репозиториумот.

2. **Login to Docker Hub:** Се врши логирање на DockerHub користејќи тајни вредности (DOCKERHUB_USERNAME, DOCKERHUB_TOKEN) дефинирани преку [GitHub Secrets](#).
3. **QEMU и Buildx:** Се поставуваат алатките за мултиплатформско градење на слики.
4. **Build и Push – Backend:** Се гради слика за Flask backend-от од ./appDemo/backend. Сликата се објавува на Docker Hub со таг nikolajkoko/image-compressor-backend:github-latest.
5. **Build и Push - Frontend:** Се гради слика за Vue frontend-от од ./appDemo/frontend. Се објавува како nikolajkoko/image-compressor-frontend:github-latest.

Овој процес овозможува секоја промена во кодот да биде автоматски рефлектирана во нови Docker слики.¹

```
name: Build and push docker image

on:
  push:
    branches: [ "master" ]
  pull_request:
    branches: [ "master" ]

env:
  BACKEND_IMAGE: nikolajkoko/image-compressor-backend
  FRONTEND_IMAGE: nikolajkoko/image-compressor-frontend
  DOCKER_TAG: github-latest

jobs:
  docker-build-and-push:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }

      - name: Set up QEMU
        uses: docker/setup-qemu-action@v3

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build and push backend image
        uses: docker/build-push-action@v6
        with:
          context: ./appDemo/backend
          push: true
          tags: ${ env.BACKEND_IMAGE }:${ env.DOCKER_TAG }

      - name: Build and push frontend image
        uses: docker/build-push-action@v6
        with:
          context: ./appDemo/frontend
          push: true
          tags: ${ env.FRONTEND_IMAGE }:${ env.DOCKER_TAG }
```

Слика 4. main.yaml

¹ Забелешка: CI/CD моментално е поставен само за CI (Continuous Integration), но основата е подготвена за надградба во целосен CI/CD процес со автоматизиран деплојмент.

Kubernetes

За оркестрација на апликацијата во кластерска околина, креирани се неколку Kubernetes манифести што ја опфаќаат целата инфраструктура – backend, frontend, базата на податоци, посебни конфигурации, сервиси и namespace.

Namespace

Сите ресурси се распоредени во посебен namespace со име **image-compressor**, што овозможува подобра изолираност, управливост и прегледност во кластерот. (**namespace.yaml**)

ConfigMaps и Secrets

- **config-map-backend**: ја дефинира портата на backend (PORT=5000).
- **config-map-db**: го дефинира името на базата image_compressor (POSTGRES_DB: image_compressor).
- **secrets**: чува лозинка и корисничко име за PostgreSQL во base64 енкодиран формат.

StatefulSet за базата

Базата на податоци **PostgreSQL** е дефинирана преку StatefulSet (**statefulset.yaml**), што овозможува стабилно и постојано складирање на податоци преку PersistentVolumeClaim. Во конфигурацијата:

- Контејнерот ја користи официјалната postgres:15-alpine слика.
- Env променливите се повлекуваат од ConfigMap и Secret.
- Податоците се зачувуваат во postgres-pvc (**pvc.yaml**).
- Можност за автоматско иницијализирање преку скрипти е оставена преку init-db-script (ако се додаде дополнително).

Deployment

deployment-backend.yaml

- Deployment со 2 пода.
- Користи image од Docker Hub: nikolajkoko/image-compressor-backend:github-latest.
- Поврзува база преку DATABASE_URL кој се гради од POSTGRES_USER и POSTGRES_PASSWORD.
- Секој под експонира порта 5000.

deployment-frontend.yaml

- Deployment со 2 пода.
- Користи image од Docker Hub: nikolajkoko/image-compressor-frontend:github-latest.
- Секој под експонира порта 80.

Service

Секој сервис е достапен внатрешно преку ClusterIP:

- postgres-service: порт 5432
- backend: порт 5000
- frontend: порт 80

Ова овозможува стабилна DNS-адреса за комуникација помеѓу компонентите.

Ingress²

Ingress контролерот (во примерот со traefik) овозможува надворешен HTTP пристап преку image-compressor.local, и ги рутира патеките:

- /api → до backend сервисот
- / → до frontend сервисот

Овој пристап го поедноставува мапирањето на ресурсите и обезбедува една влезна точка до целата апликација.

Линк до сите манифести:

<https://github.com/nikolajkoko/Image-compression-project/tree/master/appDemo/kubernetes>

² За да може апликацијата да биде достапна преку Ingress на URL адресата **http://image-compressor.local**, беше потребно локално мапирање на ова име на **localhost**.

Корисни линкови

Линк до изворниот код на Github:

- <https://github.com/nikolakoko/Image-compression-project>

Линк до Docker hub сликите:

- <https://hub.docker.com/r/nikolajkoko/image-compressor-backend>
- <https://hub.docker.com/r/nikolajkoko/image-compressor-frontend>

Други корисни линкови:

- <https://www.docker.com/>
- <https://docs.docker.com/compose/>
- <https://kubernetes.io/>
- <https://kubernetes.io/docs/reference/kubectl/>
- <https://github.com/features/actions>
- <https://flask.palletsprojects.com/en/stable/>
- <https://vuejs.org/>
- <https://www.postgresql.org/>
- <https://www.base64decode.org/>
- <https://nginx.org/en/docs/>